

Asset Change Capture + Production Pulse — Plan

Status: plan locked 2026-06-13, not yet built. Companion TODO: the [Asset Journey timeline](#) consumes this as its richest event source.

1. Problem

ArtHound is the canonical production data layer, but every sync silently destroys history: `lib/sync/differ.py` detects exactly which records changed (via `source_hash`), then the writer overwrites. The system cannot answer "*what changed since yesterday?*" — the first question a production manager asks. There is no change stream for the planned notification system to subscribe to, and no event source for the asset-journey thesis in CLAUDE.md.

2. Locked decisions (2026-06-13)

#	Decision	Choice
1	Capture scope	Full PAW — <code>replicated_assets</code> , <code>replicated_work</code> , <code>replicated_products</code> , <code>replicated_item_types</code> . Caveat accepted: product diffs are name-only until the PAW product field schema lands; the capture layer is entity-generic so products enrich automatically.
2	Storage posture	Postgres + export contract — PG stays the system of record; the event table is born warehouse-shaped (append-only, immutable, self-contained, monthly partitions); cold partitions export to Parquet in Supabase Storage (P3). No second datastore until read patterns demand it.
3	Noise policy	Capture all, filter on read — every non-suppressed field delta is persisted (<code>[IGNORE]</code> / <code>ingest_suppressed</code> already excluded upstream); the feed filters by the existing meta-bucket/display-tier classification, slots first. Future per-field opt-out = write-side filter later.
4	v1 surface	Home widget + asset history — StudioHome "Production Pulse" panel + per-asset change-history API. Full activity page deferred to the journey work.

Defaults (confirmed)

- Init/re-seed syncs emit `created` events only, no field diffs** — a re-init storm of fake updates would poison the feed. The DB-reset TODO decides whether reset emits a baseline marker.
- Feed UI is studio-side v1**; the schema is org-generic (vendor replica changes are captured), vendor UI deferred.
- The notification system consumes this table when built** — change capture is the stream, notifications become pure delivery + routing. No notification work in this plan.

3. Architecture

Postgres-first, warehouse-shaped

The trigger for leaving Postgres is **read pattern** (analytical scans over deep history), not write volume: event volume is proportional to actual production activity because the differ already filters unchanged records (~1M events/year for a busy org; tens of orgs $\approx$ tens of millions of rows/year — comfortable for partitioned append-only PG). Therefore:

- **Append-only, immutable, self-contained rows** — entity name denormalized at event time; no interpretive joins, so a row means the same thing in PG or in a Parquet file.
- **Native monthly range partitions** on `occurred_at`; partition pre-creation via the existing background-loop pattern (same shape as the nightly `sync_log` trim).
- **Deliberately NO foreign keys** to `canonical_assets` / `replicated_*` — FKs would cascade-destroy history when an entity is deleted, and history outliving the entity is the point. Tenancy is enforced by RLS, not referential integrity. (Precedent: `estimate_share_dispatches` is deliberately un-FK'd.)
- **Export lever (P3)**: partitions older than the hot window (~12 months) detach → Parquet in Supabase Storage; a manifest table records exported ranges. The journey is never lost; PG stays lean forever.
- **One read module** (`lib/changes.py`) through which every consumer reads (feed, asset history, journey, notifications). This is the swap point if deep-history reads later move to DuckDB-over-Parquet or ClickHouse. Features never query the table directly.

Capture at the differ, not DB triggers

The differ is the one place "changed" is already decided. Triggers would couple capture to Postgres (anti-warehouse), fire on no-op upsert paths, and be hard to enrich. The differ is hash-only — it never holds old values — so the capture step fetches current rows **for the changed subset only** (scales with change count, not table size), computes the field diff in Python, emits events, then the writer upserts as today. Sync already runs under `system_identity()`, so the write identity is free. The sync layer stays trigger-agnostic.

Canonical-asset principle, per tier

Asset events always carry `canonical_asset_id` (app-enforced NOT NULL for `entity_kind='asset'`); work events carry it when resolvable; product/item-type events are a tier above any asset (same reasoning as the estimate rate card) and anchor to entity identity instead.

Security

- **RLS pattern C** (own-org): SELECT via owner membership on (`owner_type`, `owner_id`); **no user write policies at all** (append-only, system-written); FORCE RLS; `sys_all` for `arthound_system`; registered in `scripts/rls_grant_audit.py`; persona-matrix checks added.
- A studio's source changes are **never visible to vendors**, dispatched or not — live change streaming cross-org would be a new sharing surface and is explicitly out of scope (a future opt-in could ride

the link, like reviews).

- Per-user visibility tiers within an org don't exist in the product yet; events carry `canonical_asset_id` so a future asset-level ACL filters in the read module without schema change.

4. Schema — `paw_change_events`

Column	Notes
<code>id</code> uuid, <code>occurred_at</code> timestamptz	PK (<code>id</code> , <code>occurred_at</code>) — partition key must be in the PK
<code>sync_run_id</code> uuid	groups a sync batch (plain value, not FK — <code>sync_log</code> is trimmed)
<code>owner_type</code> text, <code>owner_id</code> uuid	org scope, RLS anchor
<code>entity_kind</code> text	<code>asset \</code> <code>work \</code> <code>product \</code> <code>item_type</code>
<code>source_type</code> , <code>source_record_id</code> , <code>entity_name</code>	self-contained identity + display name at event time
<code>canonical_asset_id</code> uuid NULL	NOT NULL app-enforced for assets; set for work when resolvable
<code>event_type</code> text	<code>created \</code> <code>updated \</code> <code>deleted</code>
<code>changed_fields</code> text[]	field keys; slots as <code>__slots.status</code>
<code>changes</code> jsonb	<code>{field: {old, new}}</code> ; per-value truncation cap (~1KB) bounds row size
<code>slot_changes</code> jsonb	extracted slot diffs — the feed's fast path
<code>created_at</code> timestamptz	

Indexes per partition: (`owner_type`, `owner_id`, `occurred_at` desc); partial (`canonical_asset_id`, `occurred_at` desc) where `canonical_asset_id` is not null. GIN on `changed_fields` deferred until a real query needs it (insert-path cost).

5. API / UI (v1)

- `GET /api/changes` — org-scoped feed; cursor pagination on (`occurred_at`, `id`); grouped by `sync_run_id`; filters: `entity_kind`, slots-only; slot changes ranked first via the existing display-tier classification.
- `GET /api/changes/asset/{canonical_asset_id}` — per-asset history (the journey timeline's source feed).
- StudioHome "**Production Pulse**" panel: latest changes grouped by sync run.

6. Phasing

Phase	Delivers
P0 — Capture	Partitioned table + RLS/grants + differ integration (all four entity kinds) + partition-maintenance loop + guards green. No UI — history starts accumulating immediately.
P1 — Feed	<code>lib/changes.py</code> read module, both endpoints, StudioHome widget.
P2 — Enrichment	Delete events from full-sync reconciliation; source actor attribution where available (Jira changelog has per-field authors; Airtable does not); better work→canonical resolution.
P3 — Cold path	Partition detach + Parquet export to Storage + manifest + read-module fallback. The warehouse pivot point, exercised before it is urgent.

7. Risks

- **Extra SELECT per sync batch** (changed rows only) — negligible at delta sizes, measurable on large full syncs; batch the fetch.
- **Row size** — `changes` jsonb with large text fields; per-value truncation cap.
- **Field-churn noise** (formulas, rollups) — accepted by decision 3; read-side tier filtering is the control; per-field opt-out is the escape hatch if a studio's source is pathological.
- **Partition ops on Supabase** — native declarative partitioning works; maintenance runs in the existing app background loop (no `pg_cron` dependency).

8. Relationship to other work

- **Notification system (TODO):** this table is the subscribable stream; notifications become delivery + routing only.
- **Asset Journey timeline (TODO):** `GET /api/changes/asset/{id}` is one of its event sources, alongside `review_events`, the payload dispatch lifecycle, estimate shares, and acceptance records.
- **DB-reset TODO:** decides re-seed semantics (baseline marker vs silence).

9. ArtHound-principle check

- **Security first:** org-walled RLS, append-only, system-written, no new cross-org surface.
- **Canonical assets:** every asset/work event keyed to its canonical asset; product-tier events documented as the sanctioned exception (tier above assets).
- **Named fields = universal truths:** the event schema is entity-generic; field semantics stay in jsonb.
- **Trigger-agnostic sync:** capture lives in the differ path shared by login/poll/webhook syncs.